

# Building CI-CD Pipeline 3 (GitHub Container Registry - CD)

## GitHub Container Registry

Build and push docker images into the container registry

### What is GitHub Container Registry?

It is the GitHub's version of **Docker Hub (Container Registry)**

A Container Registry stores:

```
Built Docker Images
├─ sitwallet-backend:latest
├─ sitwallet-frontend:latest
└─ sitwallet-backend:v1.0
```

For this workflow, it is needed as the current GitHub actions only does the following:



Once the workflow finishes, the images would have been disappeared / destroyed - Docker Image only exists temporarily on the GitHub Actions runner.  
Images only existed on a temporary Ubuntu VM. Once workflow is done, everything on that VM is deleted.

1. Developer pushes code to GitHub
2. GitHub Actions automatically triggers
3. Backend dependencies are installed and audited
4. Frontend dependencies are installed and built
5. Docker images are created
6. Images are pushed to GitHub Container Registry (GHCR)
7. Images become available for deployment to EC2

## GitHub Container Registry

### Building and Pushing Docker Images

1. Adjust `ci.yaml` in the local repository to make it able to build and push the Docker Images

```
name: SITWallet CI/CD

# Trigger workflow whenever code is pushed to main
# or when a Pull Request is opened against main
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

# Required for pushing Docker images to GitHub Container Registry (GHCR)
permissions:
  contents: read
  packages: write

# Global variables used throughout the workflow
env:
  REGISTRY: ghcr.io

  # Backend image name
  BACKEND_IMAGE: ghcr.io/koksenglim0/sitwallet-backend

  # Frontend image name
  FRONTEND_IMAGE: ghcr.io/koksenglim0/sitwallet-frontend

jobs:
  build-and-push:

    # Name displayed in GitHub Actions UI
    name: Build and Push Docker Images
```

```
# GitHub-hosted Ubuntu runner
runs-on: ubuntu-latest

steps:

  # Step 1: Download repository source code
  - name: Checkout repository
    uses: actions/checkout@v4

#####
# BACKEND CI
#####

# Install Node.js for backend
- name: Setup Node.js for backend
  uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: npm
    cache-dependency-path: backend/package-lock.json

# Install backend dependencies
- name: Install backend dependencies
  working-directory: backend
  run: npm ci

# Check backend dependencies for vulnerabilities
# Fail pipeline if High/Critical vulnerabilities are found
- name: Run backend audit
  working-directory: backend
  run: npm audit --audit-level=high

#####
# FRONTEND CI
#####

# Install Node.js for frontend
- name: Setup Node.js for frontend
  uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: npm
    cache-dependency-path: frontend/package-lock.json

# Install frontend dependencies
- name: Install frontend dependencies
  working-directory: frontend
  run: npm ci

# Build frontend production files
# Ensures React/Vite application compiles successfully
- name: Build frontend
  working-directory: frontend
  run: npm run build

#####
# CONTAINER REGISTRY AUTHENTICATION
#####

# Login to GitHub Container Registry (GHCR)
# Uses GitHub's built-in token
- name: Login to GitHub Container Registry
  if: github.event_name == 'push'
  run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin

#####
# DOCKER IMAGE BUILD
#####

# Build backend Docker image
- name: Build backend Docker image
  run: docker build -t $BACKEND_IMAGE:latest ./backend

# Build frontend Docker image
- name: Build frontend Docker image
  run: docker build -t $FRONTEND_IMAGE:latest ./frontend

#####
# PUSH IMAGES TO CONTAINER REGISTRY
#####
```

```
# Push backend image to GHCR
- name: Push backend Docker image
  if: github.event_name == 'push'
  run: docker push $BACKEND_IMAGE:latest

# Push frontend image to GHCR
- name: Push frontend Docker image
  if: github.event_name == 'push'
  run: docker push $FRONTEND_IMAGE:latest
```

2. After that, push the changes to the GitHub repository by executing the commands:

```
git add .github/workflows/ci.yaml
git commit -m "Add Docker image push to GHCR"
git push origin main
```

3. Once you push the changes, you will be able to see the building and the pushing of the Docker Images into the GitHub Container Registry as it triggered the PUSH trigger

The screenshot shows the GitHub Actions interface for a workflow named "Add Docker image push to GHCR (200626) #5". The workflow is in a "Completed" state, indicated by a green checkmark. The left sidebar shows the "Summary" tab selected, with a list of jobs under "All jobs". The main panel displays the "Build and Push Docker Images" job, which succeeded 2 minutes ago in 1m 14s. The job steps are listed with their durations:

- Set up job: 1s
- Checkout repository: 2s
- Setup Node.js for backend: 7s
- Install backend dependencies: 2s
- Run backend audit: 1s
- Setup Node.js for frontend: 5s
- Install frontend dependencies: 4s
- Build frontend: 3s
- Login to GitHub Container Registry: 1s
- Build backend Docker image: 10s
- Build frontend Docker image: 15s
- Push backend Docker image: 11s
- Push frontend Docker image: 7s
- Post Setup Node.js for frontend: 1s
- Post Setup Node.js for backend: 0s
- Post Checkout repository: 0s
- Complete job: 0s

```
1 ▶ Run echo "****" | docker login ghcr.io -u koksenglim0 --password-stdin
8
9 WARNING! Your credentials are stored unencrypted in '/home/runner/.docker/config.json'.
10 Configure a credential helper to remove this warning. See
11 https://docs.docker.com/go/credential-store/
12
13 Login Succeeded
```

```
1 ▶ Run docker build -t $BACKEND_IMAGE:latest ./backend
8 #0 building with "default" instance using docker driver
9
10 #1 [internal] load build definition from Dockerfile
11 #1 transferring dockerfile: 160B done
12 #1 DONE 0.0s
13
14 #2 [internal] load metadata for docker.io/library/node:20-alpine
15 #2 ...
16
17 #3 [auth] library/node:pull token for registry-1.docker.io
18 #3 DONE 0.0s
19
20 #2 [internal] load metadata for docker.io/library/node:20-alpine
21 #2 DONE 0.8s
22
23 #4 [internal] load .dockerignore
24 #4 transferring context: 2B done
25 #4 DONE 0.0s
26
27 #5 [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
28 #5 resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.0s done
29 #5 sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 0B / 1.26MB 0.1s
30 #5 sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 7.67kB / 7.67kB done
```

#### Build frontend Docker image

```
1 ▶ Run docker build -t $FRONTEND_IMAGE:latest ./frontend
8
8 #0 building with "default" instance using docker driver
9
10 #1 [internal] load build definition from Dockerfile
11 #1 transferring dockerfile: 308B done
12 #1 DONE 0.0s
13
14 #2 [internal] load metadata for docker.io/library/node:20-alpine
18 #2 DONE 0.1s
19
20 #3 [internal] load metadata for docker.io/library/nginx:alpine
21 #3 ...
24
25 #4 [auth] library/nginx:pull token for registry-1.docker.io
26 #4 DONE 0.0s
27
28 #3 [internal] load metadata for docker.io/library/nginx:alpine
29 #3 DONE 0.7s
30
31 #5 [internal] load .dockerignore
32 #5 transferring context: 2B done
33 #5 DONE 0.0s
34
35 #6 [build 1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
36 #6 DONE 0.0s
37
38 #7 [stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:d565d19ef132a5834f5897f602831ad2e40a36c26c625f2f94f9b3fdf0ed292d
42 #7 resolve docker.io/library/nginx:alpine@sha256:d565d19ef132a5834f5897f602831ad2e40a36c26c625f2f94f9b3fdf0ed292d done
43 #7 sha256:9d80b52aca3c36b340585ac903125d1ea10a27752e4eded2a3eda4865897bc96 1.05MB / 4.36MB 0.2s
44 #7 sha256:696fe85c76cd21407aede3c8c09ab3102f8736ecfe8b05d20a422bb6569473fa 0B / 627B 0.2s
45 #7 sha256:d565d19ef132a5834f5897f602831ad2e40a36c26c625f2f94f9b3fdf0ed292d 10.33kB / 10.33kB done
46 #7 sha256:791fc301c0c48d31dc02800fb629c5d603a43303caad13c70cdddb555baca712 2.50kB / 2.50kB done
47 #7 sha256:784506a50a350d37a180b732dab7a9b0c5a5073cdf372257fb746de3387e3c 12.32kB / 12.32kB done
```

#### Push backend Docker image

```
1 ▶ Run docker push $BACKEND_IMAGE:latest
8 The push refers to repository [ghcr.io/koksenglim0/sitwallet-backend]
9 55b7d184221e: Preparing
10 91d4985e69f4: Preparing
11 9e346ccbee0e: Preparing
12 c2a0fd02d3da: Preparing
13 afa543f85b46: Preparing
14 e10358715ead: Preparing
15 4983b93ee796: Preparing
16 29df493baa13: Preparing
17 e10358715ead: Waiting
18 4983b93ee796: Waiting
19 29df493baa13: Waiting
20 afa543f85b46: Pushed
21 c2a0fd02d3da: Pushed
22 9e346ccbee0e: Pushed
23 55b7d184221e: Pushed
24 91d4985e69f4: Pushed
25 e10358715ead: Pushed
26 29df493baa13: Pushed
27 4983b93ee796: Pushed
28 latest: digest: sha256:cc0591c25759b499d82fdb124936788498a1e39237691785ec5467cebcaeb060 size: 1995
```

#### Push frontend Docker image

```
1 ▶ Run docker push $FRONTEND_IMAGE:latest
8 The push refers to repository [ghcr.io/koksenglim0/sitwallet-frontend]
9 13a6ca01961d: Preparing
10 f17f665274d0: Preparing
11 4b7144ac09da: Preparing
12 f95e278c8ea7: Preparing
13 fa10e24a2bc9: Preparing
14 38f56d5424ad: Preparing
15 f13022d1356b: Preparing
16 cf8c574ec244: Preparing
17 740439659ec7: Preparing
18 29df493baa13: Preparing
19 38f56d5424ad: Waiting
20 f13022d1356b: Waiting
21 cf8c574ec244: Waiting
22 740439659ec7: Waiting
23 29df493baa13: Waiting
24 fa10e24a2bc9: Pushed
25 f95e278c8ea7: Pushed
26 13a6ca01961d: Pushed
27 f17f665274d0: Pushed
28 38f56d5424ad: Pushed
29 f13022d1356b: Pushed
30 cf8c574ec244: Pushed
31 4b7144ac09da: Pushed
32 29df493baa13: Mounted from koksenglim0/sitwallet-backend
33 740439659ec7: Pushed
34 latest: digest: sha256:ebce6a724ce2d5b98d8e7198016bb1dd9a84739ff7793e5666848ecd60c564a size: 2406
```

4. After the workflow of building and pushing of the Docker Images is done successfully, you will be able to see the packages as shown below:

2216-ssd-10-main Private

main 1 Branch 0 Tags

Go to file Add file Code

**koksenglim0** Add Docker image push to GHCR (200626) 47f1557 · 4 minutes ago 11 Commits

|                           |                                        |               |
|---------------------------|----------------------------------------|---------------|
| .github/workflows         | Add Docker image push to GHCR (200626) | 4 minutes ago |
| backend                   | Fix npm audit vulnerability            | 2 hours ago   |
| docs                      | Move project files to repository root  | 3 hours ago   |
| frontend                  | Move project files to repository root  | 3 hours ago   |
| README.md                 | Move project files to repository root  | 3 hours ago   |
| docker-compose.local.yaml | Move project files to repository root  | 3 hours ago   |

**README**

# SITWallet

Secure SITWallet implementation using a Node.js/Express backend, MySQL stored procedures, and an integrated React/Vite frontend.

## Quick Start

1. Install backend dependencies:

**About**

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

**Releases**

No releases published [Create a new release](#)

**Packages 2**

- sitwallet-backend
- sitwallet-frontend

**Contributors 1**

- koksenglim0** chaocorporal

**Languages**

github.com/koksenglim0?tab=packages&repo\_name=2216-ssd-10-main

koksenglim0

Repositories 6 Projects Packages Stars

Type: All Search packages... Visibility: All Sort by: Most downloads

2 packages

- sitwallet-backend** Private Published 4 minutes ago by chaocorporal in sitwallet-backend
- sitwallet-frontend** Private Published 4 minutes ago by chaocorporal in sitwallet-frontend

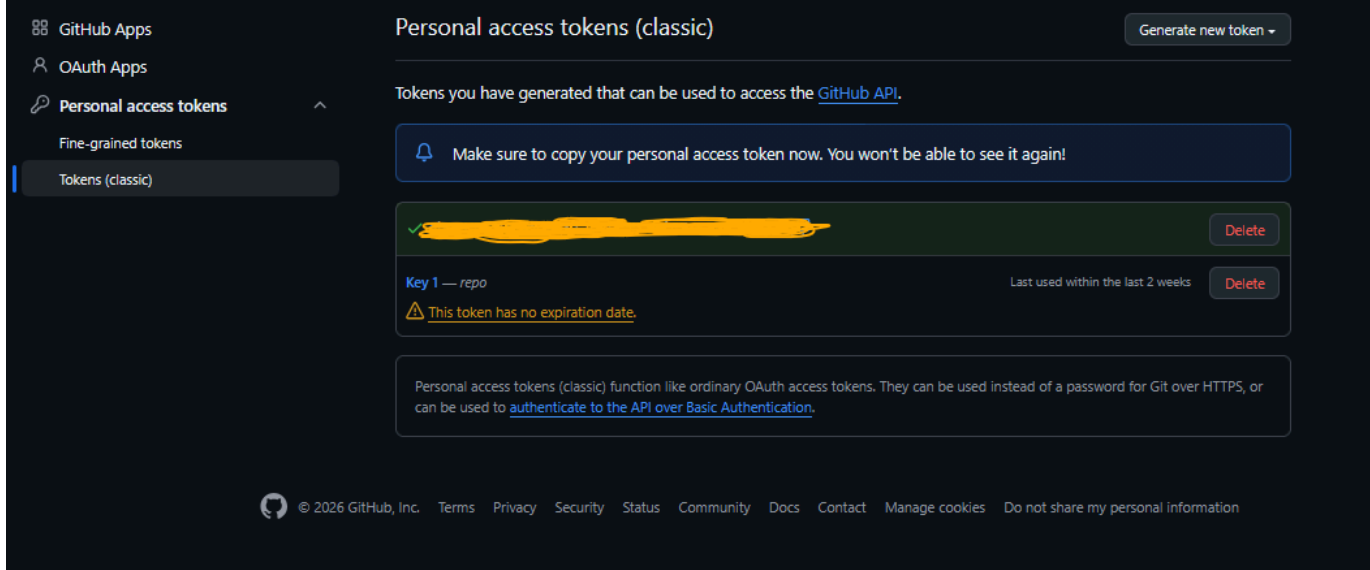
**chaocorporal**  
koksenglim0

Edit profile

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

## Testing Docker Images at GHCR

1. Before login into GHCR using the PowerShell, go to GitHub and generate a Personal Access Token from there.



2. To login, execute the command: `docker login ghcr.io -u koksenglim0`.  
When prompted for password, paste the token into the session.

```
Login Succeeded
PS C:\Users\kokse> docker login ghcr.io -u koksenglim0
Password:

Login Succeeded
PS C:\Users\kokse> docker pull ghcr.io/koksenglim0/sitwallet-backend:latest
latest: Pulling from koksenglim0/sitwallet-backend
55f8430b9f0b: Pull complete
9ed300ef6dc0: Pull complete
1bd77d06c39f: Pull complete
c5c933b034ca: Pull complete
Digest: sha256:f01d1693bf4ceb0805081d45ba37b6791f8119e847efb369a79a44ad3a388b5c
Status: Downloaded newer image for ghcr.io/koksenglim0/sitwallet-backend:latest
ghcr.io/koksenglim0/sitwallet-backend:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview ghcr.io/koksenglim0/sitwallet-backend:latest
PS C:\Users\kokse> |
```

3. After authenticating using the token, pull the images from GHCR using the commands:

```
docker pull ghcr.io/koksenglim0/sitwallet-backend:latest
docker pull ghcr.io/koksenglim0/sitwallet-frontend:latest
```

4. Create another folder in the Desktop (own preference) and create another `yaml` with the following content within the folder.

```
services:

#####
# MYSQL DATABASE
#####
mysql:

  # Use MySQL 5.7 image
  # Matches SITWallet schema requirements
  image: mysql:5.7

  # Container name displayed in Docker
  container_name: sitwallet-mysql-ghcr-test

  # Database initialization settings
  environment:

    # MySQL root password
    MYSQL_ROOT_PASSWORD: rootpassword

    # Database automatically created during startup
    MYSQL_DATABASE: sitwallet

  # Persist database data even if container restarts
  volumes:
    - mysql_ghcr_test_data:/var/lib/mysql
```

```
#####
# BACKEND APPLICATION
#####
backend:

  # Pull backend image from GitHub Container Registry
  image: ghcr.io/koksenglim0/sitwallet-backend:latest

  # Container name displayed in Docker
  container_name: sitwallet-backend-ghcr-test

  # Environment variables required by SITWallet backend
  environment:

    # Production mode
    NODE_ENV: production

    # Listen on all interfaces
    HOST: 0.0.0.0

    # Backend application port
    PORT: 5000

    #####
    # DATABASE CONNECTION SETTINGS
    #####
    DB_HOST: mysql
    DB_PORT: 3306
    DB_USER: root
    DB_PASSWORD: rootpassword
    DB_NAME: sitwallet

    #####
    # SECURITY CONFIGURATION
    #####

    # JWT signing secret
    # Generate using:
    # openssl rand -hex 32
    JWT_SECRET: 1b061f2a7156ceab102ea8cd792ef8eba693a8e510cea250cb7d298a7e4308f7

    # Data encryption key
    # Generate using:
    # openssl rand -hex 32
    APP_ENCRYPTION_KEY: a7c30a5b848c24cf0791782f8e417109be27552cb8a64078f3dc3a85436aafc7

    # Allow cookies over HTTP during testing
    COOKIE_SECURE: "false"

    # Enable CSRF protection
    CSRF_ENABLED: "true"

    # Frontend URL allowed to access backend APIs
    CORS_ORIGIN: http://localhost:8081

  # Start backend after MySQL starts
  depends_on:
    - mysql

#####
# FRONTEND APPLICATION
#####
frontend:

  # Pull frontend image from GitHub Container Registry
  image: ghcr.io/koksenglim0/sitwallet-frontend:latest

  # Container name displayed in Docker
  container_name: sitwallet-frontend-ghcr-test

  # Map localhost:8081 → container port 80
  ports:
    - "8081:80"

  # Start frontend after backend starts
  depends_on:
    - backend
```

```
#####
# DOCKER VOLUME
#####

volumes:

  # Persistent storage for MySQL database
  # Prevents data loss when container restarts
  mysql_ghcr_test_data:
```

## APP\_ENCRYPTION\_KEY

- Before running the command, execute this command first for APP\_ENCRYPTION\_KEY:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Command is for generating a valid key in the Powershell and to be used in the application.

APP\_ENCRYPTION\_KEY must be a 32-byte base64 or 64-character hex value.

Copy the value into `docker-compose.local.yaml`

```
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main> docker compose -f docker-compose.local.yaml down
[+] down 4/4
✔Container sitwallet-frontend      Removed    0.0s
✔Container sitwallet-backend      Removed    0.1s
✔Container sitwallet-mysql        Removed    0.0s
✔Network 2216-ssd-10-main_default Removed    0.2s
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main>
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main>
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main> node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
971be696c9a1e6cfe7914f107c846fe4d9664459113a7d28834fdfe93ce9c521
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main>
```

## JWT\_TOKEN

- Execute this command first for JWT token:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Command is for generating a valid key in the Powershell and to be used in the application.

JWT\_TOKEN must be a 32-byte base64 or 64-character hex value.

Copy the value into `docker-compose.local.yaml`

```
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main>
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main> node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
4197831a39e91d61d532682f41494d1bc1d5a2e34b333ec4e632a97fa5021d6e
PS C:\Users\kokse\OneDrive\Desktop\2216-ssd-10-main>
```

```
mysql_ghcr_test_data:
  type: volume
  driver: local

# Run Service
backend:
  build: ./backend
  container_name: sitwallet-backend
  environment:
    NODE_ENV: production
    HOST: 0.0.0.0
    PORT: 5000
    CORS_ORIGIN: http://localhost:8080
    DB_HOST: mysql
    DB_PORT: 3306
    DB_USER: root
    DB_PASSWORD: rootpassword
    DB_NAME: sitwallet
    JWT_SECRET: 4197831a39e91d61d532682f41494d1bc1d5a2e34b333ec4e632a97fa5021d6e
    APP_ENCRYPTION_KEY: 971be696c9a1e6cfe7914f107c846fe4d9664459113a7d28834fdfe93ce9c521
    COOKIE_SECURE: "false"
    CSRF_ENABLED: "true"
  ports:
    - "5000:5000"
  depends_on:
    - mysql

# Run Service
frontend:
  build: ./frontend
```

`docker-compose.ghcr-test.yaml`

- After inserting the values, you can test-run the images locally by running docker-compose: `docker compose -f docker-compose.ghcr-test.yaml up -d`

```
PS C:\Users\kokse\OneDrive> cd Desktop
PS C:\Users\kokse\OneDrive\Desktop> cd docker_ghcr
PS C:\Users\kokse\OneDrive\Desktop\docker_ghcr> docker compose -f docker-compose.ghcr-test.yaml up -d
[+] up 8/8
✔Image ghcr.io/koksenglim0/sitwallet-frontend:latest Pulled
✔Network docker_ghcr_default                        Created
✔Volume docker_ghcr_mysql_ghcr_test_data            Created
✔Container sitwallet-mysql-ghcr-test                 Started
✔Container sitwallet-backend-ghcr-test               Started
✔Container sitwallet-frontend-ghcr-test              Started
PS C:\Users\kokse\OneDrive\Desktop\docker_ghcr>
```



Run `docker ps` to check on the status of the containers

Install the latest PowerShell for new features and improvements! <https://aka.ms/PSWindows>

```
PS C:\Users\kokse> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
433c24ee50ad   ghcr.io/koksenglim0/sitwallet-frontend:latest   "/docker-entrypoint..." 16 seconds ago Up 15 seconds 0.0.0.0:8081->8081/tcp
ee6911d08e63   ghcr.io/koksenglim0/sitwallet-backend:latest    "docker-entrypoint.s..." 16 seconds ago Up 15 seconds 5000/tcp
9d06ec2bab76   mysql:5.7                                     "docker-entrypoint.s..." 16 seconds ago Up 15 seconds 3306/tcp, 33060/tcp
```

8. Insert the schema for the database into the test database

```
Get-Content .\001_schema_mysql57_all_procedures.sql -Raw | docker exec -i sitwallet-mysql-ghcr-test mysql -uroot -prootpassword
```

```
PS C:\Users\kokse\OneDrive> cd Desktop
PS C:\Users\kokse\OneDrive\Desktop> Get-Content .\001_schema_mysql57_all_procedures.sql -Raw | docker exec -i sitwallet-mysql-ghcr-test mysql -uroot -prootpassword
mysql: [Warning] Using a password on the command line interface can be insecure.
PS C:\Users\kokse\OneDrive\Desktop>
```

9. Attempt to access the application using `http://localhost:8081`

Try to register and login successfully into the application

The screenshot displays the SITWallet dashboard in a web browser. The dashboard features a dark blue header with the SITWallet logo and a search bar. The main content area is divided into several sections: a 'TOTAL ESTIMATED BALANCE' section showing \$0.00 for 1 active wallet, a 'Quick Actions' section with buttons for Send, Add, Exchange, and History, a 'Your Wallets' section showing a Singapore Dollar wallet with an available balance of \$0.00, and a 'Recent Transactions' section showing no transactions yet. A sidebar on the left contains navigation links for Dashboard, Send Money, Exchange, and History. The top navigation bar includes a search bar and user profile options.

10. This meant that the Frontend and Backend images in GHCR can be used after this round of testing